

Procedural Generation of Sokoban Levels

Joshua Taylor and Ian Parberry

Technical Report LARC-2011-01

Laboratory for Recreational Computing
Department of Computer Science & Engineering
University of North Texas
Denton, Texas, USA

February, 2011



Procedural Generation of Sokoban Levels

Joshua Taylor
Dept. of Computer Science & Engineering
University of North Texas
JoshuaTaylor@my.unt.edu

Ian Parberry
Dept. of Computer Science & Engineering
University of North Texas
ian@unt.edu

ABSTRACT

We describe an algorithm for the procedural generation of levels for the popular Japanese puzzle game Sokoban. The algorithm takes a few parameters and builds a random instance of the puzzle that is guaranteed to be solvable. Although our algorithm and its implementation runs in exponential time, we present experimental evidence that it is sufficiently fast for offline use on a current generation PC when used to generate levels of size and complexity similar to those human-designed levels currently available online.

Keywords

Procedural generation, Sokoban, puzzle.

1. INTRODUCTION

In puzzle games the level design can make the difference between a game that is trivially easy or completely impossible. It is difficult to find the balance between the two, where the levels are challenging but still solvable. Here we present an algorithm that automates the generation of Sokoban puzzles of a given difficulty.

Sokoban is a puzzle game played on a rectangular grid. The goal is for the player's avatar to push boxes onto marked goal squares. The challenge comes from the placement of the walls, goals and boxes and the restriction that the avatar is only strong enough to push one block at a time and cannot pull blocks at all. The simplest way of explaining it is to show a picture, for example Figure 1, which shows a level with a single box and a single goal. This figure and the other screenshots in this paper are from JSoko [4].

Culberson [3] has shown that Sokoban is PSPACE-complete, meaning that it is in a sense at least as difficult as almost any one-player game. (Most games that are hard in this sense are for two or more players.) This, together with its simple rules, makes Sokoban a challenging candidate for procedural generation of puzzle instances. Completely random

Sokoban levels are extremely likely to be unsolvable, or if they are solvable, then they are likely to be very easy. Even hand-made levels suffer from this problem unless the person making the level is an experienced Sokoban level designer.

Most other research done on Sokoban has been geared towards solving existing Sokoban puzzles (Junghanns and Schaeffer [8], Botea, Müller, and Schaeffer [2]). Some work has also been done on estimating the difficulty of a given Sokoban problem (Jarušek and Pelánek [6], Ashlock and Schonfeld [1]). Relatively little research has been done on generating new Sokoban levels (Murase, Matsubara, and Hiraga [11], Masaru, Tomoyumi, and Satoru [9]), although there are several existing generator programs (for example, those listed in [10]). Additionally, there has been some research on generating levels for other PSPACE-complete puzzle games (for example, Servais [12]).

The remainder of this paper is divided into six sections. Section 2 discusses what we want to achieve in more detail. Section 3 describes our algorithm at a high level in pseudocode. Section 4 discusses how our Sokoban generator constructs level sets. Section 5 gives some experimental data on the expected running time of our Sokoban generation algorithm. Section 6 contains the conclusion and some open problems.

The interested reader is invited to visit our Sokoban Generator webpage [14] (see Figure 2) for supplementary information. This includes some more detailed instructions for the novice on how to play Sokoban, several hundred procedurally generated Sokoban levels, a link to an open source Java implementation called JSoko on which to play-test those levels, a short video showing JSoko's solution to some of our levels, some larger color images from this paper, and the archived data that we used to generate the tables and figures in Section 5.

2. OBJECTIVES

Any procedural generation system should satisfy several criteria (from Doran and Parberry [5]): *novelty*, *structure*, *interest*, *controllability* and *speed*. All of these are highly subjective, but they are good guidelines on what a procedural generation system should accomplish. *Novelty* refers to the system's ability to generate substantially different content with each run. *Structure* means that the system is not just generating random noise. *Interest* is a hard-to-define combination of novelty and structure and is the most subjective of the criteria. *Controllability* means that the designers using

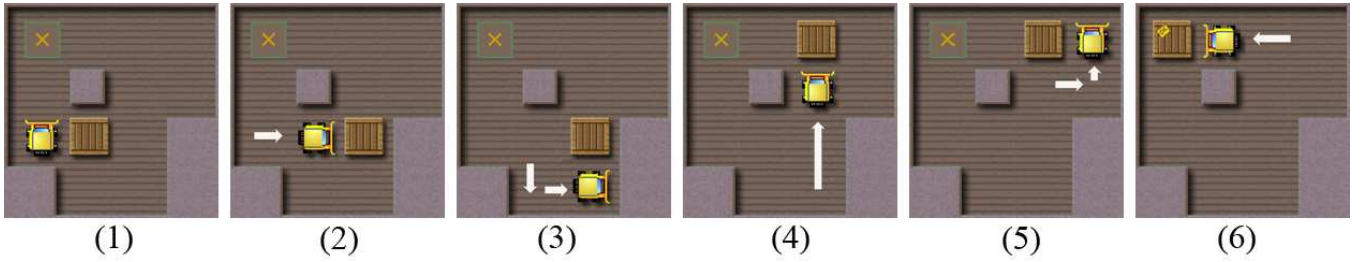


Figure 1: Solving a simple Sokoban level. The aim is to push the box to the square marked with the “X” at top left using the yellow bulldozer. The white arrows indicate player actions. The six images show, from left to right, (1) the start configuration, (2) push the box one place right, (3) reposition the player below the box, (4) push the box two places up, (5) reposition the player to the right of the box, and (6) push the box two places to the left into the final configuration.

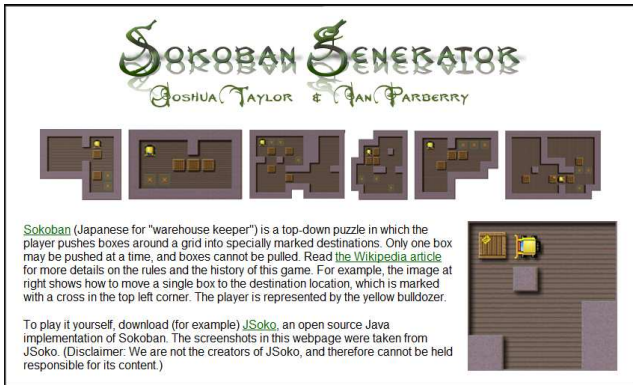


Figure 2: The Sokoban Generator webpage [14] associated with this paper.

the system have some way to make it create outputs within given design parameters. *Speed* is meant in the subjective sense: The system needs to be “fast enough”, but what that means depends on how it is being used.

Our system focusses mainly on interest. As interest depends on novelty and structure, those criteria are also important. The system is somewhat controllable, but given how specific it is, there is not much room for design limitations, and it is meant to run completely autonomously. Finally, the system can be quite slow on moderately sized levels, but is fast enough for a puzzle-a-day style game, or for generating level sets offline.

Our primary aim is to generate reasonably difficult, but not impossible, Sokoban levels. There are two reasons for this. First, these levels are the kind that are hard for a human to make, at least without a lot of experience. Second, we feel that in puzzle games, difficulty is related to interest. Of course, any game that is too difficult will become uninteresting, but making easy levels is a much simpler problem.

3. METHOD

The idea of working backwards from the goal towards the start is not new (see, for example, Takes [13]), but previously

it has only been used to *solve* existing levels. Here we use that idea to *generate* new levels. Our algorithm consists of three high-level steps, each of which will be described in more detail in its own subsection below.

1. Build an empty room.
2. Place goals in the room.
3. Find the state farthest from the goal state.

3.1 Empty Rooms

To build an empty room, we use a method somewhat similar to that of Murase, Matsubara, and Hiraga [11]. We begin by choosing a width and height for the level. This is done by simply picking a random number within a user-specified range. The level is then partitioned into a grid of 3×3 blocks. Each block is then filled in using a randomly chosen and randomly rotated or flipped template. The templates consist of a 3×3 pattern of walls and floors surrounded by a border of blanks, walls and floors (see Figure 3). The borders cause neighboring templates to overlap. A non-blank tile must match any pattern it overlaps, whether it is placed before or afterwards.

This overlap helps to create interesting levels by preventing some bad configurations from being generated. For example, the pattern consisting of a single wall in the middle surrounded by a ring of floor will become a large dead-end unless there are at least two floor tiles adjacent to each other and that pattern. Since the templates are randomly rotated and flipped before being placed, this is very easy to enforce by simply placing two adjacent floor tiles in the border of that template and leaving the rest blank.

If the generator places blocks in such a way that it cannot fill in one of the cells with any of the available templates, it will discard that attempt and start over. The run time of this step is very small compared to the rest of the algorithm, so even throwing away several partial room shapes does not create any noticeable loss of speed.

Finally, there are some post-processing checks to make sure the level will work well with the remaining steps. Any level that fails one or more of these tests is discarded.

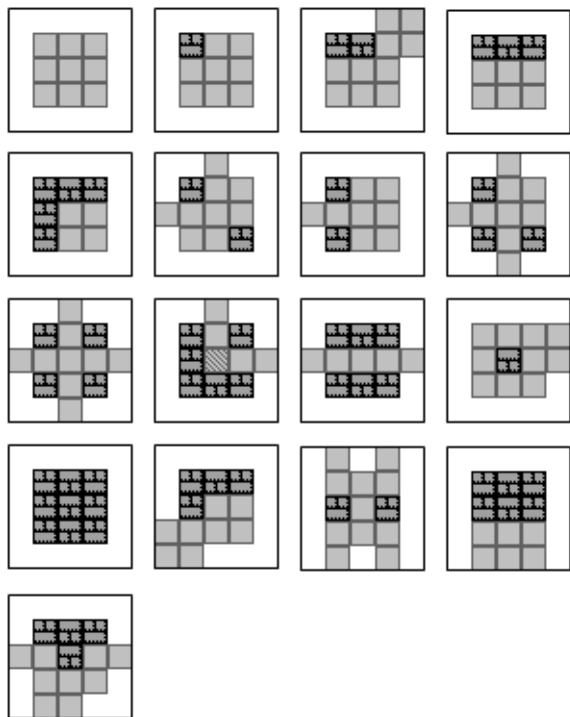


Figure 3: Templates used to design an empty room.

- The level is checked for connectivity. There should be one contiguous section of floor. There is one special case here. The templates that allow the player to pass through, but will not allow a box to pass, are checked as if there was a wall tile separating the two sides. This only affects this check, and that tile is counted as a floor tile in all other cases.
- Any level that has a 4×3 or 3×4 (or larger) section of open floor is discarded. By observation, such levels tend to make levels with very bushy, but not very deep state spaces. This makes it very hard to generate the level, but not much harder to solve it.
- The level must have enough floor space for the planned number of boxes, plus the player and at least one empty space.
- If the level contains any floor tiles surrounded on three sides by walls, it is discarded. This is a somewhat aesthetic choice, but such tiles are either obviously dead space if there is no goal there, or an easy place to get boxes out of the way otherwise, so we think it improves the quality of the resulting levels somewhat.

3.2 Placing Goals

Goal placing is done by brute force, trying every possible combination of goal positions. This is admittedly very inefficient. Many human made levels place the goals in certain patterns, such as a rectangle of contiguous goals, but by doing a brute force search for the best places to put the goals, some obvious patterns emerge.

One pattern that seems to hold for most, but not quite all, of the levels generated so far is that the goals are touching either a wall or another goal. Whether forcing this would provide a significant speed-up, or a significant drop in the quality of the resulting levels has not yet been investigated.

Our generator uses a timer that checks it has exceeded its allotted time. If it has, it will terminate and return the best result so far. To help ensure that that result is something interesting, even if not the best, the positions for the goal crates are checked in random order. This is done by creating a shuffled list of the empty spaces on the board.

3.3 Farthest State

For each placement of the goals the system finds the farthest state from that goal state, that is, the state with the longest shortest path from itself back to the goal. Over all goal states, the farthest farthest state is returned as the output of the generator. Thus, the distance from the goal state to the start state is the metric by which we judge the resulting levels, as well as influencing the algorithm used to search the state space. The definition of *distance* is crucial. In Sokoban there are four common distance metrics. The simplest is just the move count, incremented every time the avatar moves. As a measure of the difference between states, this does not work very well. Just making a large labyrinth with only one obvious solution will still give a high distance, but will be fairly trivial in the end.

The number of box pushes, incremented each time the avatar moves into a square containing a box, is not much different than the move count. A level that required the player to push boxes down long hallways would give a high score, but again would not be difficult, just tedious.

The box lines metric is more interesting. It counts how many times the player pushes a box, but any number of pushes of the same box in the same direction only count as a single box line. From our observations, the number of box lines corresponds fairly well with the difficulty of the resulting level. We are currently using the box line metric in our generator.

The last metric is box changes. It counts how many times the player stopped pushing one box, in any direction, and began pushing another. This may be an even better measure of difficulty, and may improve the overall speed of the generator, but it is more difficult to implement.

Any metric except for the move count allows us to abstract out the avatar position. Instead of keeping up with which square the avatar is in, we keep up with which group of contiguous floor squares it could reach. This abstraction provides a significant decrease in the time it takes to generate the set of further states.

All of this is done in reverse compared to how Sokoban is played. The reason for this is to prevent the generator from having to consider invalid moves. Any state reachable when moving in reverse will be solvable when played normally.

Unfortunately, none of the usual search algorithms are suitable for this problem. The most obvious way to find the far-

the state is to use a breadth-first search, returning the last state found, but since moves in Sokoban are not reversible, the only way to prevent repetitions is to store a list of all visited nodes. For Sokoban, or any other PSPACE-complete problem, this will quickly fill up the available memory. Iterative deepening is unsuitable for similar reasons. Informed searches, like A* or IDA*, are unsuitable because the target is very vaguely defined, meaning we have no clear indication when to stop the search. To get around these problems, we use a form of iterative deepening twice, trading off the high memory requirements for a somewhat slower algorithm.

```

proc Go(goal) ≡
  startSet := MakeStartSet(goal);
  resultSet := startSet;
  depth := 1;
  do
    prevSet := resultSet;
    resultSet := Try(startSet, resultSet, depth);
    if resultSet = ∅ then exit fi;
    depth := depth + 1;
  od;
  Go := (prevSet, depth).

```

```

proc Try(startSet, prevResults, depth) ≡
  resultSet := Expand(prevResults);
  tempSet := startSet;
  for i := 1 to depth do
    resultSet := resultSet − tempSet;
    tempSet := Expand(tempSet);
  od;
  Try := resultSet.

```

MakeStartSet takes the goal state and places the player into each available contiguous floor area. **Expand** takes a set of states and returns the set of states one step farther. What those states are depends on which metric is being used, which is why the choice of metric has such an impact on the running time. **Go** is almost a standard iterative deepening algorithm. It takes the goal, calls **MakeStartSet** to set things up and then calls **Try** one depth at a time. What is different here is the end condition. **Go** calls **Try** until it fails and then returns the previous set of results. **Try** takes the starting set, the previous results and a target depth. It then calls **Expand** on the previous results. Then it starts over expanding the start set, subtracting that from the results. What is left after the target depth has been reached is all of the nodes that can be reached in **depth** steps, but no sooner. If **depth** is too great, this will be an empty set.

4. GENERATING LEVEL SETS

Our generator returns the set of all levels that are as far from the goal state as possible. This can be anywhere from one to a few hundred levels, and some are obviously better than others. We have an additional layer over the generator that attempts to select a good level from those generated and then collects the results of several runs into a level set. Additionally, it makes an attempt to reject levels that are much too easy, or are too similar to levels already in the level set.

Finally, when the target number of levels has been reached, it attempts to sort them by some measure of difficulty.

The questions of what is *better* and what is *not good enough* both rely on the same, rather arbitrary, measure. We take the candidate level and give it a score based on a number of factors. To begin with, the score is $100 \cdot (\text{pushes} - \text{number of sibling levels} + 4 \cdot \text{lines} - 12 \cdot \text{boxes}) + \text{Random}(0, 300)$, where **pushes** is the number of box pushes in the solution, **sibling levels** is the number of other levels the generator found at the same depth, **lines** is the number of box lines in the solution, **boxes** is the number of boxes in the level and **Random** is just a random number between the given values. While this is, again, fairly arbitrary, the rationale is that both more pushes and more lines make the level more difficult, while levels with many sibling levels seem to be less interesting, just by observation. The number of boxes is subtracted not because more boxes makes the level less interesting, but because the number of lines needs to exceed the number of boxes by a certain factor for the level to have a better chance of being a good level. The random factor is mainly there to break ties.

Some other checks are made after getting the base score. Any trapped box is worth -100000 points, which is almost guaranteed to get the level rejected. Any box touching a wall is worth -150 points, a box touching the player is worth 50 points, and a box touching another box is worth 30 points. Finally, a goal area touching a goal area is worth 30 points. These constants can be adjusted by the individual designer to suit his or her intuition about features possessed by good Sokoban levels. Any level with a final score of 0 or less is rejected. The base score is quite a bit higher than the scores for most of the various other checks though, so not many levels are rejected at this point. This same score is then used to choose the best level from those generated, assuming any are left.

Once a level has passed all of the other tests, the program checks to see whether or not it is too similar to another level already in the set. Currently, this just removes the player and checks for exact matches for all of its rotations and reflections. This still generates a few levels that a human would consider too similar, so there is still more work to be done here.

Assuming no other level is too similar, the level is added to the set. When the set gets to target size, it is sorted by difficulty and written to a file. The measure of difficulty we currently use is $\text{lines} \cdot \log \text{lines} + \log \text{time} - \text{lines} / \text{pushes}$. This is based on our observations that the number of box lines is the most important factor. **time** is the time taken to generate that level, in seconds, and is mainly used as a tie breaker. The **lines/pushes** factor is small correction that favors levels with shorter box lines rather than long corridors.

5. EXPERIMENTAL RESULTS

We have implemented and tested our new algorithm for the automatic generation of Sokoban levels. Figure 9 contains some screenshots of sample levels generated. The reader is invited to visit our Sokoban Generator webpage [14], where he or she can download some of our level sets and try them out.

Size	Moves	Time
1×2	26	< 1 sec
2×2	48	1.9 sec
2×3	60	16 sec
3×3	73	128 sec

Table 1: The average experimental running time for the generation of 2-box puzzles of various sizes, with the corresponding average number of moves needed to solve them. The averages were computed over 10 random samples each. The number of moves were calculated from the generated data by JSoko.

Our algorithm is certainly suitable for offline use in a level-a-day style game. In practice it can generate several levels over the course of a day depending on how much CPU power it is given and how large the desired levels are. The theoretical run time for the generation of one level is roughly

$$b \binom{s}{b}^2,$$

where b is the number of boxes and s is the number of empty spaces. This is feasible for a fairly small number of boxes that might occur in practice. We have been able to generate levels with 4 boxes within a 2×3 level outline within a few hours.

We ran experiments measuring the average run-time generating 10 random puzzles of each size 1×2 , 2×2 , 2×3 , and 3×3 . See Figure 4 for an indication of the relative sizes of these levels. All of these results are from runs on an Intel i7 3.2 GHz quad-core processor with hyperthreading. Our system is not written to make use of the extra cores, but we run several independent copies of the code simultaneously, relying on the operating system to place each copy on a separate processor core.

Tables 1 and 2 show in Column 3 the experimental running time required to generate 2-box and 3-box puzzles respectively, averaged over 10 samples for each entry. These data are depicted pictorially in Figure 5 with an exponential trendline. Tables 1 and 2 also show in Column 2 the average number of moves required to solve the generated puzzles using the autosolver in JSoko, set to “move optimal with best pushes”. These data are depicted pictorially in Figure 8 with a quadratic trend line.

Table 3 shows in Column 3 the average experimental running time for 2×2 puzzles (which have 36 cells). See the level at top right of Figure 4 to get some idea of the size of the puzzle. These data are depicted pictorially in Figure 7 with an exponential trendline. Table 3 also shows in Column 2 the average number of moves required to solve the generated puzzles using the autosolver in JSoko, set to “move optimal with best pushes”. These data are depicted pictorially in Figure 6 with a quadratic trend line.

Levels with a single box are generally uninteresting. Levels with 2 boxes can be generated very quickly, usually within a few seconds, but tend to be very easy. At 3 boxes the levels start to get slightly more interesting, and can still be

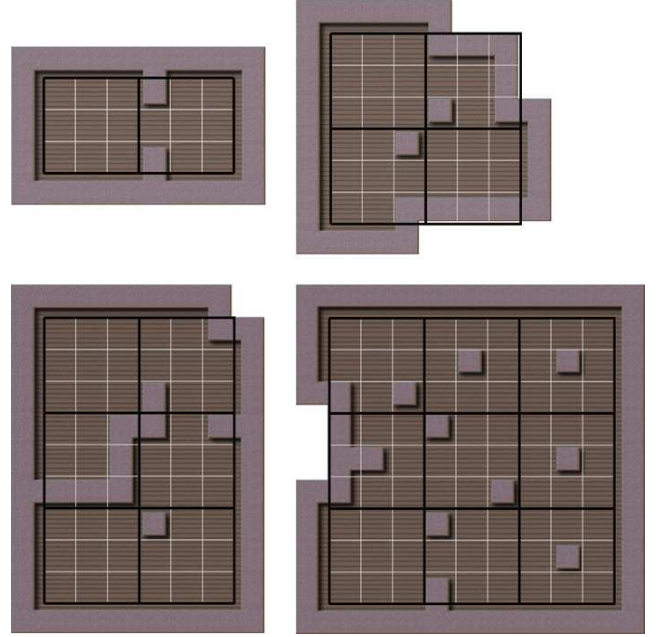


Figure 4: Examples showing the relative sizes of 1×2 , 2×2 , 2×3 , and 3×3 Sokoban levels. Note that, for example, a 2×2 grid such as the one in the top-right corner has 4 subgrids (outlined in black) of 9 cells each (outlined in gray).

Size	Moves	Time
1×2	38	58 sec
2×2	69	2.7 min
2×3	98	1.1 hr
3×3	115	24.5 hr

Table 2: The average experimental running time for the generation of 3-box puzzles of various sizes, with the corresponding average number of moves needed to solve them. The averages were computed over 10 random samples each. The number of moves were calculated from the generated data by JSoko.

Boxes	Moves	Time
2	48	1.9 sec
3	69	2.7 min
4	100	3.4 hr
5	109	26 hr

Table 3: The average experimental running time for the generation of 2×2 puzzles, with the corresponding average number of moves needed to solve them. The averages were computed over 10 random samples each. The number of moves were calculated from the generated data by JSoko.

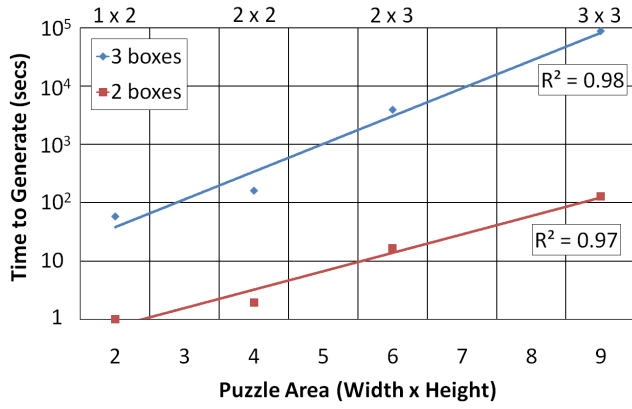


Figure 5: The average experimental running time in seconds for the generation of puzzles with 2 and 3 boxes versus puzzle area. The corresponding grid sizes are at the top. Note that the vertical axis has a logarithmic scale. The line is a best-fit exponential.

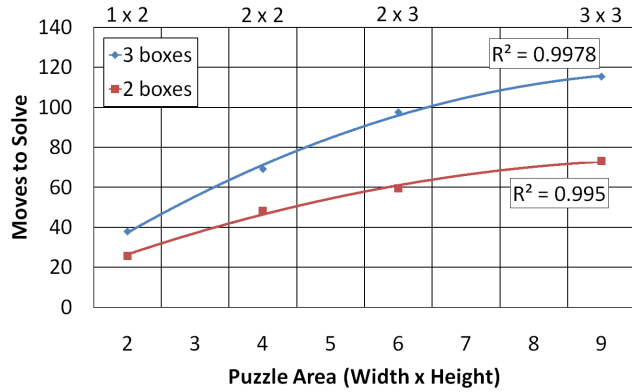


Figure 6: The average experimental number of moves needed to solve 2 and 3-box puzzles versus puzzle area. The corresponding grid sizes are at the top. The line is a best-fit quadratic.

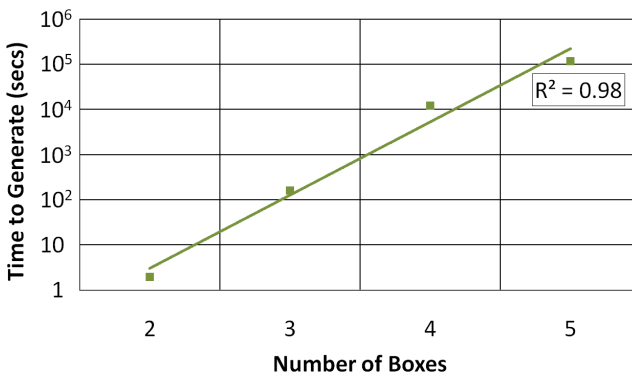


Figure 7: The average experimental running time in seconds versus number of boxes for the generation of 2×2 puzzles. Note that the vertical axis has a logarithmic scale. The line is a best-fit exponential.

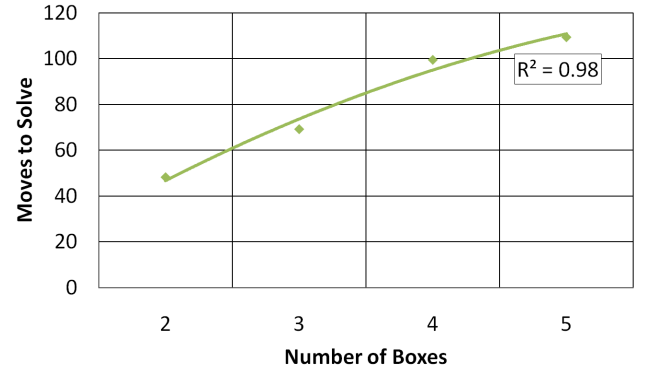


Figure 8: The average experimental number of moves versus number of boxes for the generation of 2×2 puzzles. The line is a best-fit quadratic.

generated within a few minutes. Levels with 4 or more boxes can be very interesting, and difficult, but take substantially more time to generate. Using the timer feature, we can force the generator to return the best result after a given time period. Using a time limit of 4 hours, we have generated levels with 5 and 6 boxes that appear interesting and difficult (for example Figure 9 includes some 5-box puzzles).

Using just iterative deepening, the algorithm runs several times faster, but uses much more memory. On some levels, the program crashed after consuming over 1.5GB of memory. Using our algorithm, the same levels never exceeded 40MB of memory.

6. CONCLUSION AND FUTURE WORK

While we found the puzzles that we generated “interesting”, we provide no justification for this claim in this paper, although we do invite the reader to try for themselves by visiting our Sokoban Generator webpage [14]. We plan to gather data from play-testing in the next phase of this research, and we will report the results in a later paper. Some research into what makes a level interesting and what makes it difficult is needed, though some research on these questions has already been done (for example, Ashlock and Schonfeld [1], Jarušek and Pelánek [7]).

There are several parts of our program that need additional research. The location of the goals within the level is currently done with a brute force search, simply trying every combination of locations. Some method of discarding certain combinations would significantly cut down on the run time. A first step would be to investigate whether placing goals along walls would be a good trade between speed and missed possibilities.

The algorithm we use for generating the level once the goals are placed trades off speed for a much lower memory requirement. As low as the memory requirements have become, there is some room to trade some back, though it is not obvious where to make such trades.

Whether box lines or box changes is better correlated to difficulty is an open question, though the answer is probably

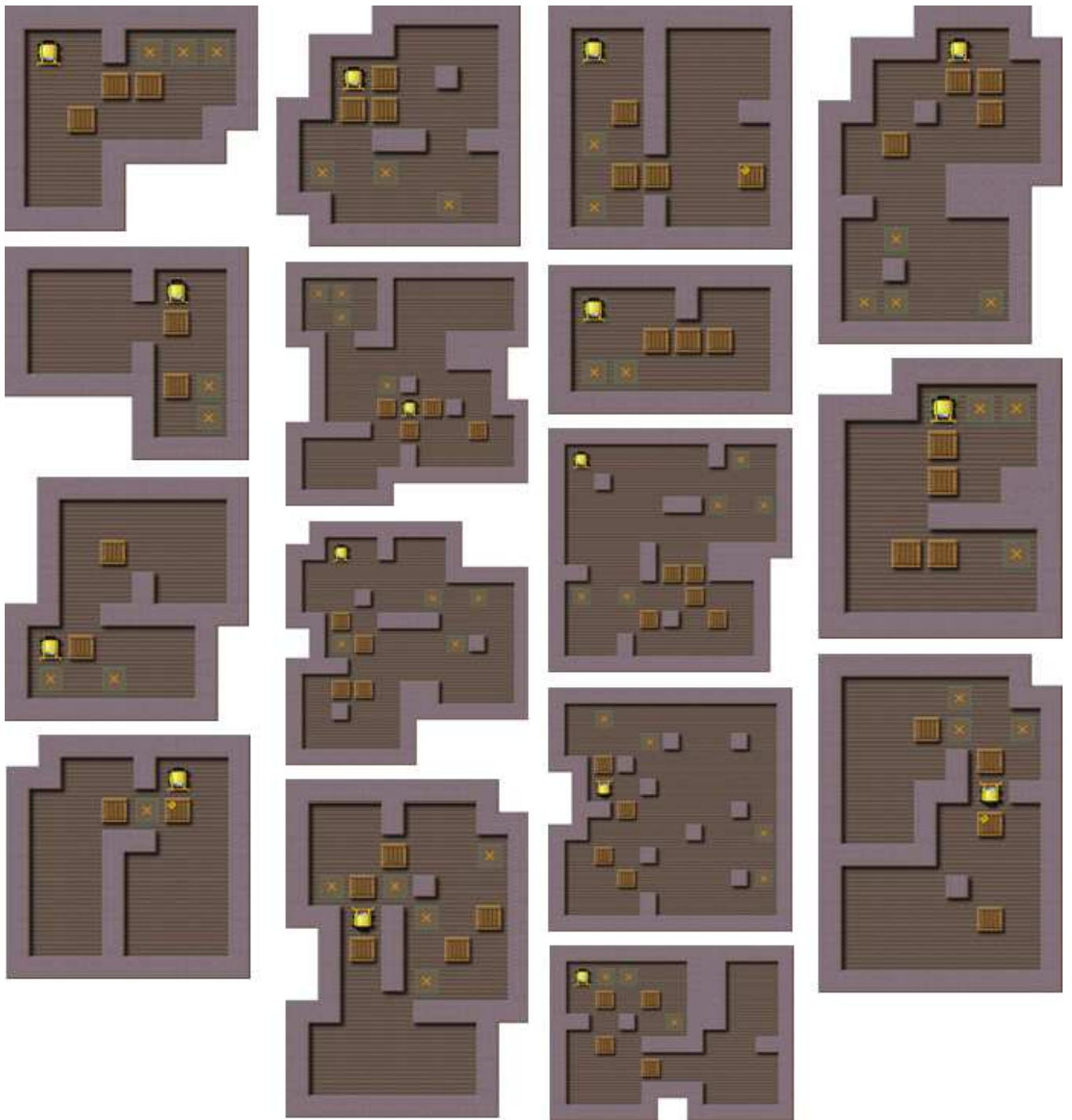


Figure 9: Some levels of varying difficulty created by our generator.

somewhat subjective. Using the states one box change farther away may be faster overall than using those one box line away. They would be slower to generator at each step, but there would usually be much fewer steps. Whether this results in a significant improvement needs to be tested.

Currently, our system can only handle a maximum of about 6 boxes. Increasing this number would require some means of breaking the level down into subsections, but solving individual puzzles is not as interesting as solving one well-integrated puzzle, so doing this in a non-obvious way is a challenge.

Finally, this research could conceivably be extended the procedural generation of levels for many different puzzle games. Reducing the amount of game-specific information needed by the generator will take some effort.

7. REFERENCES

- [1] D. Ashlock and J. Schonfeld. Evolution for automatic assessment of the difficulty of Sokoban boards. In *Proceedings of the IEEE Congress on Evolutionary Computation*, pages 1–8, 2010.
- [2] A. Botea, M. Müller, and J. Schaeffer. Using abstraction for planning in Sokoban. In J. Schaeffer, M. Müller, and Y. Björnsson, editors, *Computers and Games*, volume 2883 of *Lecture Notes in Computer Science*, pages 360–375. Springer, 2003.
- [3] J. Culberson. Sokoban is PSPACE-complete. In *Proceedings of the International Conference on Fun with Algorithms*, pages 65–76, 1998.
- [4] B. Damgaard, H. Marxen, and M. Meger. JSoko. <http://sourceforge.net/projects/jsokoapplet/>.
- [5] J. Doran and I. Parberry. Controlled procedural terrain generation using software agents. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(2):111–119, 2010.
- [6] P. Jarušek and R. Pelánek. Difficulty rating of Sokoban puzzle. In *Proceedings of the Fifth Starting AI Researchers’ Symposium*, 2010.
- [7] P. Jarušek and R. Pelánek. Human problem solving: Sokoban case study. Technical Report FIMU–RS–2010–01, Faculty of Informatics, Masaryk University Brno, 2010.
- [8] A. Junghanns and J. Schaeffer. Sokoban: A challenging single-agent search problem. In *Proceedings of the IJCAI Workshop on Using Games as an Experimental Testbed for AI Research*, pages 27–36, 1997.
- [9] O. Masaru, K. Tomoyumi, and K. Satoru. A method of automatic creation of goal-area in Sokoban maps. *Joho Shori Gakkai Shinpojiumu Ronbunshu*, pages 67–74, 2003.
- [10] M. Mühendisi. Sokoban Level Generators (A to Z). <http://www.erimsever.com/sokoban7.htm>.
- [11] Y. Murase, H. Matsubara, and Y. Hiraga. Automatic making of Sokoban problems. In N. Foo and R. Goebel, editors, *PRICAI’96: Topics in Artificial Intelligence*, volume 1114 of *Lecture Notes in Computer Science*, pages 592–600. Springer, 1996.
- [12] F. Servais. *Finding Hard Initial Configurations of Rush Hour with Binary Decision Diagrams*. M.Sc. Thesis, Université libre de Bruxelles, Faculté des Sciences. 2005.
- [13] F. Takes. *Sokoban: Reversed Solving*. Bachelor’s Thesis, Leiden University. 2007.
- [14] J. Taylor and I. Parberry. Sokoban Generator. <http://www.eng.unt.edu/ian/research/sokoban/>.